

# Supervisório 3D e gêmeo digital da célula de paletização R-01

Motoman GP12 · visualização tridimensional em navegador, dirigida pelos sinais reais da linha, com fonte alternável entre célula e simulador.

| ROBÔ                 | CONTROLADOR             | CÓDIGO                     | LINGUAGEM        |
|----------------------|-------------------------|----------------------------|------------------|
| Yaskawa Motoman GP12 | Siemens S7 · Modbus TCP | 3.555 linhas · 32 arquivos | TypeScript · SCL |

## 1 O que o sistema é

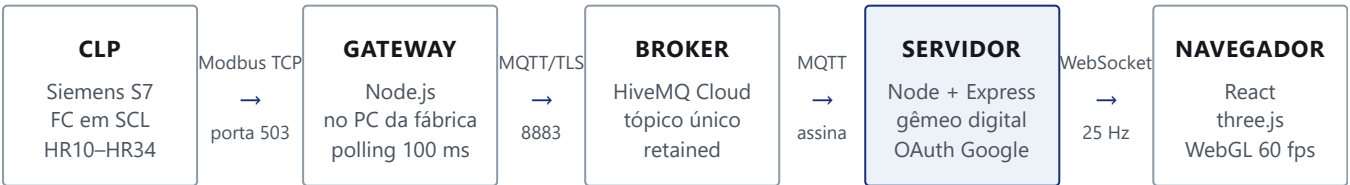
Uma página web que desenha, em três dimensões, a célula de paletização R-01 em funcionamento: o robô, a esteira de entrada, a balança de roletes, a seladora, os dois paletes e a pilha crescendo caixa a caixa. Abre em qualquer navegador, sem instalação, com login restrito ao domínio da empresa.

A decisão central do projeto está numa frase: **o movimento do robô não é lido, é gerado**. O controlador do robô não publica seus ângulos de junta ao CLP, e obter isso exigiria mexer no programa do robô. Em vez disso, o sistema conhece a cinemática real do GP12 e usa os **sinais discretos da linha** — peça em posição na balança, contador de caixas, bits de lado ativo, presença de palete — para decidir *quando* cada trecho do movimento acontece. Os eventos são reais; o que se interpola é apenas o caminho entre eles.

Isso caracteriza um **gêmeo digital orientado a eventos**, e não uma animação decorativa nem um espelho de telemetria. A distinção tem consequência prática: a tela nunca mostra a célula produzindo quando ela está parada, porque não há nada que a faça andar sozinha.

## 2 Arquitetura

Cinco camadas, cada uma com uma responsabilidade única e um protocolo de fronteira explícito. Nenhuma camada conhece a implementação da seguinte.



O sentido do fluxo é único: da célula para a tela. O supervisório **observa e não comanda** — o bloco no CLP só lê tags e escreve no DB de Modbus, e não há caminho de escrita da web para o processo. Essa é uma escolha

de segurança, não uma limitação pendente.

### 3 Camada 1 — CLP Siemens S7

Um bloco de função em **SCL** ( FC "07 – SUPERVISORIO" , 275 linhas) publica o estado da célula nas holdings livres do Modbus que já existia no projeto. Ele foi escrito para ser carregado **uma vez**: exporta tudo o que a célula tem de útil, inclusive o que o supervisorio ainda não consome. Word sobrando no Modbus é de graça; outra intervenção no CLP, não.

MAPA DE HOLDINGS — 25 WORDS USADAS, HR35–99 LIVRES

| HR    | TIPO  | CONTEÚDO                                                                                   |
|-------|-------|--------------------------------------------------------------------------------------------|
| 0–3   | —     | Integração da balança Toledo, <b>intocada</b> ; HR4–9 de folga proposital para ela crescer |
| 10    | bits  | Robô: run, servo, master job, falha, remoto, home, lado ativo, vácuo, encaixotamento       |
| 11    | bits  | Célula e segurança: sensores, presença de palete (SP4/SP5), portas, barreiras, emergência  |
| 12–14 | Int   | Passos das sequências de inicialização e dos dois lados                                    |
| 15–22 | Int   | Quantidade do lado ativo, posição no mosaico, camada, alturas, deslocamentos               |
| 23–25 | Int   | Alarmes do robô e da balança, pressão de ar em centésimos de bar                           |
| 26    | Int   | <b>Heartbeat</b> — incrementa a cada ciclo                                                 |
| 27–30 | misto | Paletes produzidos, causas da emergência, mosaico ativo, torre e vácuos                    |
| 31–34 | Int   | Status do robô, de cada lado, e modo auto/manual                                           |

#### DUAS DECISÕES DE PROJETO QUE EVITAM PARADA DE LINHA

**Porta 503, não 502.** A 502 já tem um cliente: a balança, que escreve nas holdings 1 e 2. Uma instância de MB\_SERVER atende uma conexão, e duas instâncias na mesma porta são intercambiáveis — o gateway, reconectando, poderia ocupar a vaga da balança. O life bit dela congelaria, o alarme subiria e a linha pararia. Porta própria é vaga reservada.

**Heartbeat como prova de vida.** O Modbus responde exatamente igual com o CPU em STOP. Só um contador andando prova que o programa executa — e é por isso que HR26 existe e é ele, não a conexão, que define o estado plcOk .

### 4 Camada 2 — Gateway Modbus → MQTT

Processo Node.js que roda no PC da fábrica. Lê HR0–HR34 a cada 100 ms com `modbus-serial` , decodifica bit a bit para um objeto tipado, e publica no broker. Três características valem registro, todas nascidas de defeito observado em operação:

- **Publica quando muda, e uma vez por segundo de qualquer forma.** Publicar só na mudança economiza rede, mas cria um engano: célula parada com CLP rodando não muda nada, o gateway emudece, e o supervisorio — que exige mensagem recente — anuncia falha onde não há. Silêncio não é falha, e o sinal de vida a cada segundo faz o critério de frescor significar o que deve: o gateway está vivo.

- **Quando o Modbus cai, ele diz.** A publicação vive dentro da leitura bem-sucedida, então um timeout emudecia o gateway e a última mensagem retida no broker continuava afirmando `plc0k: true` indefinidamente — uma mentira com cara de verdade. Hoje, sem contato, o último retrato é republicado com `plc0k: false` e hora nova.
- **Uma leitura Modbus de cada vez.** O timeout é de 1,5 s (35 holdings num S7 que também atende a balança, em rede de fábrica, não caberiam em meio segundo). Sem trava de reentrância, o polling de 100 ms empilharia dezenas de requisições na mesma conexão, e o congestionamento que causou o primeiro timeout causaria todos os seguintes.

## Empacotamento para o PC da fábrica

Esse PC não alcança o registro do npm, e a rede da empresa barra executáveis em pasta compartilhada — o que inclui os binários dentro de `node_modules`. A solução é o **esbuild** gerando dois arquivos de JavaScript puro, com todas as dependências embutidas: `index.cjs` e `testa-modbus.cjs`. Nenhum `.exe`, `.dll` ou `.node`; o PC precisa apenas ter o Node.js. A receita é um script do `package.json`, versionada junto com o código que empacota.

## 5 Camada 3 — Broker MQTT

---

HiveMQ Cloud, TLS na porta 8883, um tópico por célula, mensagens **retained** — quem assina recebe imediatamente o último estado conhecido, sem esperar o próximo evento. A credencial vai separada da URL de propósito: senha com `@`, `:`, `/` ou `#` não sobrevive dentro de uma URL, e não é a senha que deve ceder.

Um tópico paralelo terminado em `/TESTE` é usado para validação, com um CLP sintético que reproduz cenários difíceis — troca de lado, recuo de falha, emergência, retirada de palete, sensor travado. Todas as correções deste projeto foram verificadas nele antes de ir ao ar, e nunca no tópico de produção.

## 6 Camada 4 — Servidor web

---

Node.js com **Express** servindo o cliente compilado, **ws** para o WebSocket de estado a 25 Hz, e **cookie-session** com **OAuth 2.0 do Google** restringindo o acesso ao domínio corporativo. O *upgrade* do WebSocket passa pela mesma sessão do cookie — não há porta lateral para o fluxo de dados.

### Duas fontes, um contrato

O servidor mantém duas fontes que emitem exatamente o mesmo objeto de estado: o **simulador**, que gera um ciclo completo por conta própria, e a **fonte real**, que traduz o MQTT. O cliente não sabe a diferença, o que significa que o simulador é um ambiente de desenvolvimento e demonstração legítimo, e não um caminho de código paralelo que apodrece.

A escolha da fonte é **por navegador**, guardada num `WeakMap` indexado pela conexão. Quem está de olho na célula em produção não pode ter a tela trocada porque outra pessoa, noutra sala, abriu o simulador para demonstrar. O padrão é **REAL**: quem abre um supervisão quer ver a célula, e um robô se movendo bonito no lugar de dados ausentes seria pior que uma tela vazia.

## 7 O gêmeo digital

---

É o núcleo técnico do projeto e a parte que não vem de biblioteca nenhuma.

### Cinemática

As dimensões vêm da folha de dados do GP12. A posição de cada caixa no palete é resolvida por **cinemática inversa** para os eixos L e U, com o eixo S saindo do ângulo no plano; a orientação da caixa vem do padrão do mosaico, não do braço. Entre pontos há interpolação ponto a ponto com perfil de velocidade (**Ptp**), e a entrada na pilha usa **movimento linear** cartesiano — é como o robô real assenta a caixa, e evita a trajetória em arco que atravessaria a pilha vizinha.

### O padrão catavento

Cada palete recebe 32 caixas: quatro grupos de quatro por camada, duas camadas. As sequências não são espelhos automáticas entre os dois lados — cada uma foi conferida na cena e corrigida pelo operador, e vive em quatro tabelas explícitas no código. Mudar o padrão é editar tabela, não reescrever lógica.

### Quem manda no movimento

Um único sinal governa a caixa da entrada: **peça em posição na balança**. Ligado, há caixa nos roletes esperando o robô; ao cair, o robô a retirou e o braço parte para o palete; o incremento do contador encerra o transporte. A pilha desenhada é reconstituída pelas mesmas tabelas do padrão, de modo que a contagem na tela é igual à do CLP por construção — inclusive quando a célula recua para o múltiplo de quatro após uma falha.

#### MEDIÇÃO ANTES DE IMPLEMENTAÇÃO

Três sinais que seriam a escolha natural para dirigir a animação estão **mortos** no dado real, medidos em 106 mensagens do tópico de produção: **VC1 – VACUO OK GARRA**, **ROBO – CAIXA EM INDEXADOR** e **SFC2 – INDEXADOR CAIXA AVANÇADO** permanecem em zero permanentemente. Amarrar a lógica a qualquer um deles congelaria a cena. Por isso a autoridade é da balança, que alterna de verdade.

## 8 Camada 5 — Cliente 3D

---

**React 18** com **TypeScript**, empacotado por **Vite 5**, e **three.js** através de `@react-three/fiber` e `@react-three/drei`.

A geometria é **procedural**, não malha de CAD: o corpo do robô, a esteira, a balança de roletes, a seladora e as caixas são construídos por código a partir de primitivas, com o braço inferior curvo saído de uma extrusão sobre curva Catmull-Rom, e os rótulos (logotipo, marcas) desenhados em `CanvasTexture`. Isso mantém o pacote em torno de 1 MB comprimido, em vez dos dezenas de megabytes de um STEP importado, e torna cada detalhe ajustável por parâmetro.

Uma decisão de desempenho atravessa o cliente inteiro: o estado que chega a 25 Hz não vira *state* do React. Ele é escrito numa *ref*, e cada componente da cena o lê dentro de `useFrame`, interpolando na taxa do monitor. Sem isso, haveria 25 reconciliações por segundo numa árvore de centenas de objetos. A consequência arquitetural é clara: **o servidor cronometra, o cliente interpola** — inclusive as animações de chegada da caixa pela esteira e de saída do palete na empilhadeira, cujo progresso vem pronto do servidor.

## 9 Quadro de tecnologias

### DEPENDÊNCIAS DE PRODUÇÃO E FERRAMENTAL

| CAMADA        | TECNOLOGIA                 | VERSÃO     | PAPEL                                                 |
|---------------|----------------------------|------------|-------------------------------------------------------|
| CLP           | SCL / Structured Text      | TIA Portal | Bloco de função que publica o estado no Modbus        |
| Gateway       | modbus-serial              | 8.0        | Cliente Modbus TCP                                    |
| Gateway · web | mqtt                       | 5.10       | Publicação e assinatura no broker                     |
| Gateway       | esbuild                    | 0.21       | Empacotamento em arquivo único sem executáveis        |
| Servidor      | Node.js                    | 20         | Runtime das duas pontas fora do navegador             |
| Servidor      | Express                    | 4.19       | HTTP, estáticos e rotas de autenticação               |
| Servidor      | ws                         | 8.18       | WebSocket de estado a 25 Hz                           |
| Servidor      | cookie-session             | 2.1        | Sessão assinada, compartilhada com o WebSocket        |
| Servidor      | OAuth 2.0 Google           | –          | Login restrito ao domínio corporativo                 |
| Cliente       | React                      | 18.3       | Estrutura da interface e da cena                      |
| Cliente       | three.js                   | 0.168      | WebGL: geometria, materiais, sombras                  |
| Cliente       | @react-three/fiber         | 8.17       | three.js declarativo em React                         |
| Cliente       | @react-three/drei          | 9.114      | Controles de órbita, grade, utilidades                |
| Build         | Vite                       | 5.4        | Bundler e servidor de desenvolvimento                 |
| Todo          | TypeScript                 | 5.6        | Tipagem estrita ponta a ponta, contrato compartilhado |
| Dev           | tsx · concurrently · aedes | –          | Execução direta de TS, orquestração, broker local     |
| Implantação   | Docker · Render            | –          | Imagem e hospedagem do servidor web                   |

## 10 Volume de código

LINHAS POR MÓDULO, SEM DEPENDÊNCIAS

| MÓDULO                  | RESPONSABILIDADE                                     | LINHAS       |
|-------------------------|------------------------------------------------------|--------------|
| server/simulator.ts     | Cinemática, tabelas do catavento, roteiro, simulador | 525          |
| server/fonte-mqtt.ts    | Gêmeo digital dirigido pelos sinais reais            | 500          |
| client/scene/Cell.tsx   | Cena 3D: célula, esteira, paletes, animações         | 379          |
| client/scene/Robot.tsx  | Geometria e articulação do GP12                      | 297          |
| plc/07_SUPERVISORIO.scl | Bloco de função do CLP                               | 275          |
| gateway/clp-falso.ts    | CLP sintético para validação de cenários             | 273          |
| client/ui/Panel.tsx     | Painel de status e comandos                          | 262          |
| gateway/index.ts        | Leitura Modbus e publicação MQTT                     | 228          |
| server/index.ts         | HTTP, WebSocket, autenticação, fontes                | 208          |
| shared/types.ts         | Contrato entre servidor, cliente e gateway           | 198          |
| demais módulos          | Ferramenta, estilos, testes de conexão, entrada      | 410          |
| <b>Total</b>            | <b>32 arquivos versionados</b>                       | <b>3.555</b> |

## 11 Princípios que o código segue

Cinco regras foram destiladas de defeitos reais e valem como critério de revisão para quem continuar o projeto:

- **Nível e borda não se confundem.** Quase todo sinal da célula é nível; evento é borda. Ler o vácuo como nível fazia o braço oscilar sem sair do lugar, e a segunda borda de um mesmo apanhar fazia o braço recuar no meio do caminho até o paleta.
- **Silêncio não é falha, e ausência de dado não é dado.** Ausência de mensagem tem de ser distinguível de ausência de mudança — daí o sinal de vida no gateway e o critério de frescor no servidor.
- **Nunca desenhar o que não se viu acontecer.** A pilha só aparece pelos eventos observados. Chutar que um paleta parado está cheio pintou 32 caixas onde havia quatro; mostrar menos do que existe é falta de informação, mostrar caixa inventada é mentira, e num supervisorio mentira é pior.
- **O que aponta o braço e o que se desenha são coisas diferentes.** A contagem que decide em que paleta o robô trabalha vem só dos bits de lado. A contagem desenhada tem memória por lado. Misturar as duas levava o braço ao paleta errado.
- **Previsão tem prazo de validade.** A animação da caixa chegando pela esteira é uma previsão; quem confirma que há caixa nos roletes é a balança. Sem prazo, uma previsão não confirmada deixava uma caixa parada na balança para sempre.

## 12 Implantação

---

O servidor web é publicado por imagem **Docker** na **Render**, com segredos em variáveis de ambiente — credenciais do Google, segredo de sessão, domínio autorizado e acesso ao broker. O gateway roda no PC da fábrica, iniciado pelo Agendador de Tarefas do Windows com log em arquivo. O repositório é reproduzível a partir do clone: instalação por `package-lock.json`, verificação de tipos, build do cliente e geração do pacote do gateway, todos verificados em clone limpo.

## 13 Limites conhecidos

---

- **Sinais mortos no CLP.** Os três citados na seção 7 precisam ser verificados na célula. Enquanto não alternarem, não podem dirigir nada.
- **Mosaico fixo em código.** HR29 já transporta qual dos sete padrões está ativo, mas o desenho ainda usa as tabelas do padrão corrente. Com outro produto, a contagem ficaria certa e as posições erradas até esse consumo ser implementado.
- **Pilha só a partir do observado.** Abrindo a tela no meio de um turno, o palete do lado que o robô não está atendendo aparece vazio até a próxima troca de lado. Há sinais candidatos a preencher esse vão, ainda de semântica não confirmada.
- **Descarte de peça reprovada.** O caminho de descarte está modelado na cena mas desativado, aguardando validação da regra na célula.